Week 3 Lab Handout

API Design I — Resources, Verbs, and REST Principles

Time: 110 minutes **Bring: laptop, API_NEEDS.md, lecture notes**

Before you start: today you're translating, not inventing

Every endpoint you write today should trace back to a specific line in your `API_NEEDS.md`. If you find yourself designing an endpoint that doesn't map to any need statement, stop — either you missed a need, or you're building something nobody asked for. Week 2 did the hard work of figuring out what your partner needs. Today is about giving each of those needs a real, well-formed shape: a URL, an HTTP verb, and a purpose.

Keep the lecture's REST rules open while you work:

- **URLs are nouns.** `/tasks`, not `/getTasks`.
- **HTTP verbs carry the action.** GET reads, POST creates, PUT/PATCH updates, DELETE removes.
- **Status codes carry the outcome.** Never return `200 OK` for something that failed.
- **One level of nesting is usually enough.** `/users/{id}/tasks`, not three levels deep.

Part A — Quick Setup (10 min)

1. Open your `API_NEEDS.md` from Week 2. Count your needs statements — you should have 3–6.
2. Create a new file, `ENDPOINT_LIST.md`, at your repo root.
3. Set up a table with four columns: **Method | Path | Purpose | Maps to Need.** You'll fill this in during Part B.

Part B — Draft Your Endpoint List (35 min)

For **every** needs statement in `API_NEEDS.md`, write one endpoint. Some needs may require more than one endpoint (e.g., a need to “read active tasks” might need both a list endpoint and a single-item endpoint) — that's normal, not a mistake.

Worked example (from today's lecture)

Method	Path	Purpose	Maps to Need
GET	/tasks?status=active&sort=dueDate	Return active tasks sorted by due date	“Dashboard needs to read active tasks with due dates, in order to display an upcoming-deadlines widget.”

Notice what this row does: the path is a noun, the verb matches the action (reading), and the last column proves this endpoint didn't come out of nowhere.

As you draft, keep asking

- Is this a noun, or did a verb sneak in?
- Does the HTTP method actually match what's happening (read vs. write)?
- Am I nesting this under a parent resource because it's genuinely owned by it, or should this be a query parameter instead?
- Would I recognize this endpoint's purpose without re-reading the needs statement?

Minimum requirements

- **At least 5 endpoints total.**
- **At least one write endpoint** (POST, PUT/PATCH, or DELETE) — not everything can be read-only, even if most of your needs turned out that way in Week 2.

Part C — Peer Review: REST Convention Audit (15 min)

Swap `ENDPOINT_LIST.md` with **another team — not your ring partner.** You're not checking whether their API meets *their* partner's needs (you don't know their context well enough for that); you're checking whether it follows the rules from today's lecture.

The audit checklist — use this on the endpoint list you receive

- [ ] **Nouns, not verbs.** Every path is a resource name, not an action (`/tasks`, not `/createTask` or `/getTasks`).
- [ ] **Correct HTTP verb.** GET only reads. Anything that changes data uses POST/PUT/PATCH/DELETE.
- [ ] **Consistent pluralization.** Not `/task` in one row and `/tasks` in another.
- [ ] **Reasonable nesting.** No more than one level deep.
- [ ] **Path vs. query parameters used sensibly.** Identifiers in the path, filters and sorting as query parameters.
- [ ] **No caller-specific logic in the API.** A flag like `?dashboard=true` describing *who's asking* is a red flag.

Write your feedback directly as comments or a short list at the bottom of their file. Be specific — “row 3 has a verb in the path” is useful; “looks good” is not.

Part D — Revise Based on Feedback (25 min)

Get your `ENDPOINT_LIST.md` back from your reviewers. Go through every comment and either:

- **Fix it.** Most feedback should result in a real change.
- **Push back, briefly.** If you disagree with a comment, add a one-line note explaining why you kept it as-is. Disagreeing is fine — silently ignoring feedback is not, and your instructor will be checking for visible revision either way.

This is the step the rubric is actually checking under “evidence of peer review incorporated” — a file that's identical before and after peer review is a red flag, not a sign your first draft was perfect.

Part E — Finalize (15 min)

Read through your finished `ENDPOINT_LIST.md` one more time, end to end. For each row, confirm:

1. It maps to a real Week 2 need (not something you invented while designing).
2. It follows every rule on the audit checklist above.
3. The “Purpose” column would make sense to someone who has never seen your app.

Commit and push:

```
git add ENDPOINT_LIST.md
git commit -m "Week 3: endpoint list + peer review revisions"
git push
```

(Still no branch-naming convention to follow — that starts once you're working out of the template repository your instructor hands you.)

This week's deliverable checklist

- [ ] Every Week 2 needs statement has at least one corresponding endpoint
- [ ] At least 5 endpoints total, with at least one write verb (POST/PUT/PATCH/DELETE)
- [ ] Endpoint list swapped with another team (not your ring partner) for review
- [ ] Reviewer feedback addressed — fixed or explicitly explained, not ignored
- [ ] ENDPOINT_LIST.md committed and pushed

Rubric (25 pts)

Criterion	Points
Correct REST conventions (nouns, verbs, pluralization)	8
Endpoints trace back to Week 2 needs statements	5
Appropriate use of path vs. query parameters	6
Evidence of peer review incorporated	6

Common problems and quick fixes

Problem	Fix
A needs statement doesn't cleanly become one endpoint	That's normal — split it into two (a list endpoint and a single-item endpoint, for example). Note both in the “Maps to Need” column
You can't find a write endpoint anywhere in your needs	Re-check Week 2 for anything involving “create,” “update,” or “change” — if genuinely everything is read-only, flag this to your instructor rather than inventing a fake write endpoint just to hit the minimum
Your reviewer's feedback seems wrong	Push back in writing (Part D) rather than ignoring it silently — a documented disagreement is fine, silence is not
You're not sure if something should be nested or a query parameter	Ask: “does this only exist in the context of the parent?” If yes, nest it. If it's filtering an independent resource, use a query parameter
Peer review partner is also your ring partner by accident	Swap with someone else — the point is a reviewer without context on your specific partnership, so they're auditing the rules, not your relationship

Next week: turning this endpoint list into a real, machine-readable OpenAPI contract — the design work you did today becomes something you can actually build against.